AI FOR CODING — Module Rubric

Nov 10, 2026 · @Ali Motawea

In two weeks, each team of four ships a production-grade system of the size a student team would normally build by hand in a year, by directing coding agents (Claude Code and Codex) that plan, build, test and review each other's work. Every day adds one new concept and one new layer of the same system, so the concepts are learned by using them.

Module at a glance

Item	Detail
Duration	2 weeks intensive: 10 teaching days, about 6 contact hours a day (60 h) plus build time
Format	Each day: 1 h concept + live demo, 4 h guided build on the team's system, 1 h review and checkpoint
Teams	4 students per team, one shared repository; every student drives agents every day
Prerequisites	Programming in one language, Git basics, HTTP and databases at an introductory level
Coding agents	Claude Code (CLI and IDE) and OpenAI Codex (CLI and cloud)
Stack (reference)	TypeScript monorepo: Expo mobile app, Next.js web dashboard, Node API, PostgreSQL, Claude API for in-product AI
Accounts per student	GitHub; a Claude plan or API key with Claude Code access; a ChatGPT or OpenAI plan with Codex access
Course site	AI FOR CODING on LabVerse: one interactive lab per day (Lab 01 is live)

Tool features change quickly: the module leader re-checks every tool command and plan in the week before the course starts.

Learning outcomes

By day 10 every student can:

1. **Explain how coding agents work:** the agent loop (gather, plan, act, observe, verify), the context window, tools and permissions.
2. **Engineer the context agents work from:** prompts with context, goal, limits, example and "done when"; a PRD; architecture decision records; `CLAUDE.md` and `AGENTS.md` instruction files.
3. **Extend the agents:** write reusable skills, custom commands, subagents and hooks, and connect tools through MCP, including one MCP server the team writes itself.
4. **Orchestrate several agents:** run subagents and parallel sessions on separate git worktrees, hand work between Claude Code and Codex, and use one agent to review the other's pull requests.
5. **Build AI into the product:** use the Claude API with tool use, structured outputs, streaming and retrieval, and measure the feature with an evaluation set.
6. **Automate delivery with agents:** run agents headless in CI for review, tests, documentation and releases.
7. **Stay the engineer:** approve commands safely, read every diff, catch insecure or wrong AI code, and explain any part of the system in their own words.

The capstone system

Each team builds one system across all 10 days and deploys it on day 10. The reference project is **CampusOne**, a campus super-app: students book study rooms, follow courses, chat with classmates, pay fees and ask an AI study assistant; staff manage rooms, courses and announcements from a web dashboard. A team may propose another domain on day 2 if its PRD meets the same checklist.

The checklist below is what makes it a one-year-scale project. "Pass" is the minimum for a passing capstone; "Target" is what an excellent team ships.

Area	Pass	Target
Clients	Mobile app (Expo) and web dashboard (Next.js)	Both clients, offline-friendly mobile, responsive dashboard
Users and roles	Sign-up, sign-in, 3 roles (student, staff, admin)	Plus SSO-style sign-in, invitations, per-role permissions tested

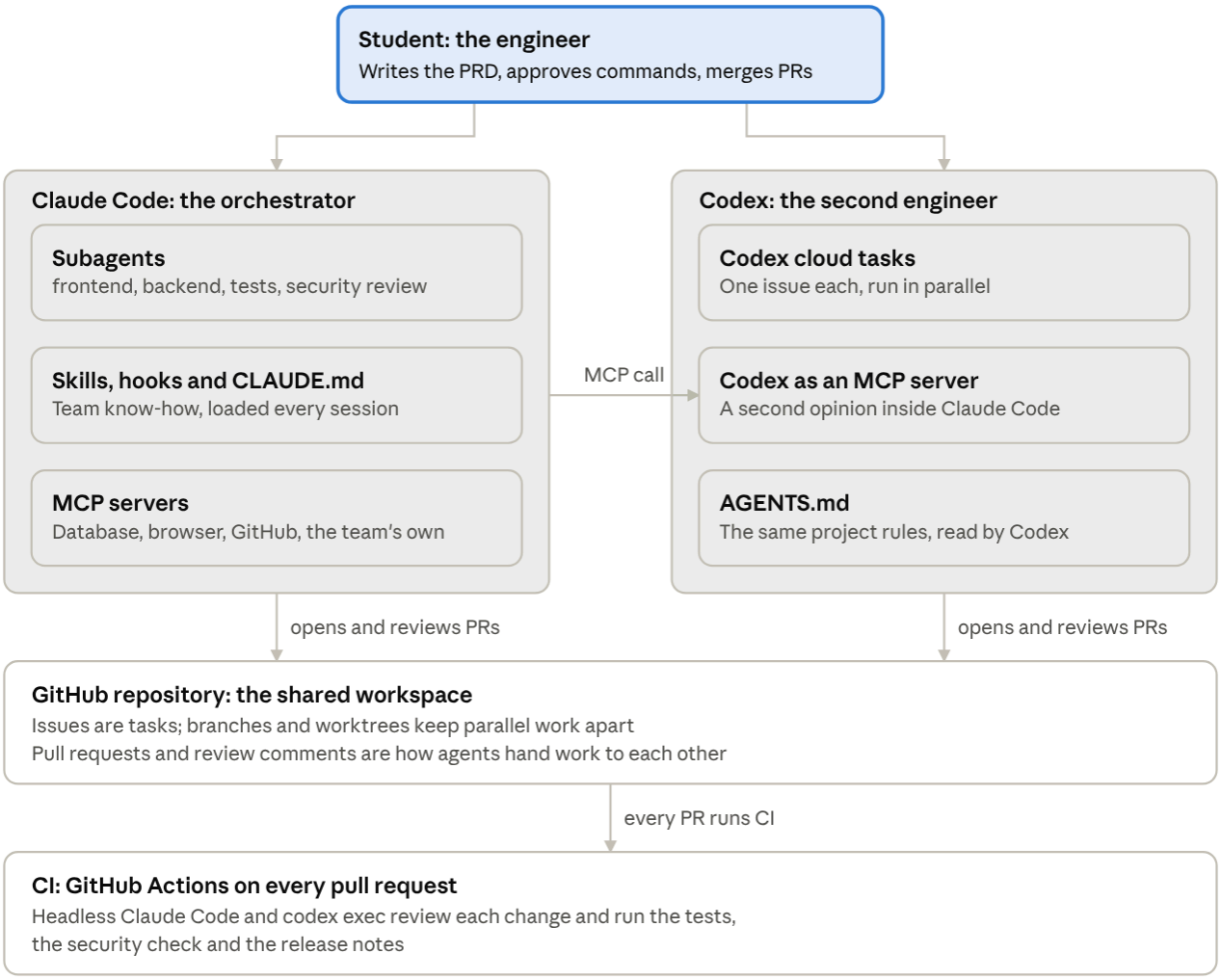
Area	Pass	Target
Domain features	4 complete user journeys from the PRD	8+ journeys, e.g. booking, courses, chat, payments, announcements, analytics
API	40+ documented endpoints, validated input	80+ endpoints, OpenAPI spec generated and published
Data	Relational schema with migrations and seed data	20+ tables, indexes, backups and a restore that was tested
Real time	Live updates on one feature	Chat and live booking status over WebSockets
AI in the product	Claude-powered assistant with tool use over the system's own data	Plus retrieval over course material, streaming, structured outputs and an evaluation set of 30+ cases
Integrations	One external service (e.g. email)	Payments in sandbox mode, push notifications, calendar export
Language and access	English and Arabic, right-to-left layout	WCAG 2.2 AA checked on the main journeys
Tests	Unit and API tests run in CI, 60% coverage of core logic	80%+ core coverage plus end-to-end tests of every journey
Delivery	CI on every pull request; one deployed environment	Staging and production, automatic agent review on every pull request, release notes
Operations	Error logging	Logs, metrics and alerts on a dashboard; a load test of the busiest endpoint
Security	Secrets out of the repo, auth on every private endpoint	A threat model, a dependency audit and an agent-run security review with fixes merged
Documentation	README, PRD, architecture overview	Plus ADRs, API docs, runbook and a user guide

How the agents work together

Students learn three ways for agents to communicate, in this order: **subagents** inside one Claude Code session (day 5), **Claude and Codex through MCP and pull requests** (day 6), and **agents in CI** reacting to every pull request (day 9). The repository is the shared

memory: an agent hands work on by opening an issue or a pull request, and the other agent answers with a review.

Agents talk through MCP calls and through the repo's issues and PRs



agent communication · student, Claude Code, Codex, repo, CI

The student is the only one who approves risky commands and merges; every agent change reaches `main` through a reviewed pull request and green CI.

Day-by-day plan

Each day introduces one concept and uses it straight away to add a layer to the team's system. Day 5 and day 10 end with a demo.

Day	New concept	Added to the system	Deliverable at end of day
1	How coding agents work: the agent loop, context window, permissions; prompting for code (Lab 01)	Monorepo skeleton, CI that runs on every pull request, first `CLAUDE.md` and `AGENTS.md`	Repo with green CI; each student's first merged agent-written PR
2	Plan before code: PRDs, plan mode, the agent interviewing you, ADRs, issues as agent tasks	PRD, architecture overview, data model, milestones broken into 30+ issues	Approved PRD and issue board
3	Tests as guardrails: test-driven work with agents, "done when" in every task	Database, auth and roles, core API endpoints with tests	15+ endpoints, tests passing in CI
4	Extending the agent: skills, custom commands, hooks (format, lint, test on every edit), permission rules	Team skills such as `add-endpoint` and `new-screen`; web dashboard first screens built with them	3+ team skills and 2+ hooks committed and used
5	Multi-agent I: subagents (frontend, backend, tests, reviewer) and parallel sessions on git worktrees	Mobile app core journeys built in parallel streams	Milestone 1 demo: two journeys working end to end
6	Multi-agent II: Claude and Codex together; Codex cloud tasks on issues; cross-agent pull-request review; Codex as an MCP server inside Claude Code	Payments (sandbox), notifications, chat; every PR reviewed by the other agent	All PRs carry an agent review and a human approval
7	Tools through MCP: database, browser (Playwright), GitHub and docs servers; writing the team's own MCP server	End-to-end browser tests; a project MCP server (e.g. seed data, feature flags); admin analytics	Team MCP server in use; E2E tests for 4 journeys

Day	New concept	Added to the system	Deliverable at end of day
8	AI inside the product: Claude API, tool use, structured outputs, streaming, retrieval, prompt caching, cost; evaluation sets	AI study assistant that acts on the system's own data	Assistant live, with an evaluation set of 30+ cases and a score
9	Agents in automation: headless Claude Code and `codex exec` in GitHub Actions for review, security review, docs and release notes	CI/CD to staging and production, observability, agent-run security review with fixes	Production deployment; security review report
10	Hardening and handover: performance, accessibility, Arabic and right-to-left, final review	Load test, accessibility fixes, runbook and user guide	Final demo, code freeze, individual viva

Marking rubric

The module is marked out of 100 on nine criteria. Criteria 1–8 are marked per team; criterion 9 is marked per student. Each criterion is placed in one of four levels, then given a mark inside that level's band.

#	Criterion	Weight	Evidence	Excellent (85–100%)	Good (70–84%)	Satisfactory (50–69%)	Insufficient (below 50%)
1	Product scope and function	20	Live demo, deployed system, capability checklist	Meets the Target column in most areas; all journeys work in production	Meets Pass everywhere and Target in several areas	Meets Pass in most areas; some journeys incomplete	Below Pass in several areas; core journeys broken
2	Architecture and code quality	10	Repository, ADRs, architecture overview	Clear module boundaries, consistent conventions, decisions recorded and followed	Sound structure with minor inconsistencies	Works, but duplication and unclear boundaries	Tangled code no one on the team can explain

#	Criterion	Weight	Evidence	Excellent (85–100%)	Good (70–84%)	Satisfactory (50–69%)	Insufficient (below 50%)
3	Testing and verification	10	CI history, coverage report, E2E runs	80%+ core coverage, E2E for every journey, tests written before or with the code	60%+ coverage and E2E for main journeys	Tests exist but are thin or often skipped	Few or failing tests; "done" never verified
4	Context engineering	12	PRD, `CLAUDE.md` / `AGENTS.md`, skills, hooks, issue history	PRD drives every milestone; instruction files kept current; 3+ skills and hooks clearly reduce rework	Good PRD and instruction files; some skills and hooks used	PRD and instruction files exist but go stale; little reuse	No usable PRD; prompts written from scratch each time
5	Multi-agent orchestration	13	Session logs, worktree branches, PR trail, MCP config	Subagents and parallel streams used deliberately; Claude and Codex hand work and review each other; team MCP server in use	Several agents used well, with some manual glue	One agent does most work; multi-agent use is token-level	Single-agent use only
6	Review discipline and security	10	PR reviews, security report, threat model	Every PR has agent and human review; risky commands denied; security findings fixed	Most PRs reviewed; main findings fixed	Reviews shallow; known issues left open	Unreviewed merges; secrets or open endpoints found
7	AI in the product	10	Assistant demo, evaluation set and scores, cost notes	Assistant uses tools over real data, retrieval and streaming; eval of 30+ cases with a measured score and cost per request	Assistant with tool use and an eval set	Assistant works but is not evaluated	No working AI feature
8	Delivery and operations	5	Pipelines, environments, dashboards, runbook	Staging and production, automatic agent review in CI, logs, metrics and alerts	Deployed with CI/CD and logging	Deployed manually	Not deployed

#	Criterion	Weight	Evidence	Excellent (85–100%)	Good (70–84%)	Satisfactory (50–69%)	Insufficient (below 50%)
9	Individual understanding	10	Viva (15 min), daily build log, contribution record	Explains and changes any part of their own work without an agent; reasons about agent output critically	Explains their own work well; minor gaps	Explains their work only in general terms	Cannot explain work committed under their name

Moderation: a student who scores Insufficient on criterion 9 may have their share of the team mark reduced by up to half, on evidence from the viva and the contribution record.

Deliverables and checkpoints

Deliverable	Due	Who	Feeds criteria
Daily build log: prompts that worked, agent mistakes caught, decisions taken (half a page)	End of every day	Each student	9
Daily checkpoint: the day's deliverable from the plan, shown to a TA	End of every day	Team	1–8
PRD, architecture overview and issue board	Day 2	Team	2, 4
Team agent toolkit: `CLAUDE.md`, `AGENTS.md`, skills, hooks, subagent definitions	Day 4, updated to day 10	Team	4, 5
Milestone 1 demo	Day 5	Team	1, 5
Team MCP server	Day 7	Team	5
AI assistant with evaluation set and results	Day 8	Team	7
Security review report and threat model	Day 9	Team	6
Final system: deployed URL, repository at the code-freeze tag, runbook, user guide	Day 10, 12:00	Team	1–8

Deliverable	Due	Who	Feeds criteria
Final demo (15 min) and individual viva (15 min)	Day 10 afternoon	Team / each student	1, 9

Daily checkpoints are formative: they earn no marks on their own, but the TA's notes are the evidence used for criteria 4–6 and 9.

AI use and integrity rules

Agents are expected to write most of the code. What is assessed is how well students direct, check and understand that work.

- **You own every line you merge.** "The agent wrote it" is never an answer in a review or the viva.
- **Every pull request** has an agent review and a human approval from a teammate who did not write it.
- **Never paste secrets** (API keys, passwords, personal data) into prompts, instruction files or code; use environment variables and the provider's secret store.
- **No force-pushes or deployments by an agent** without a human approving the command.
- **New dependencies** are checked on the package registry before installing; agents sometimes invent package names.
- **Agent session logs and the daily build log are kept** for the whole module; they are evidence for criteria 5 and 9.
- **Spending limits:** each team sets a usage cap on its Claude and OpenAI accounts on day 1 and records any overrun in the build log.
- **Viva without agents:** the viva is done without AI tools; students explain and change their own code live.

Open questions

- [ ] Is the module 2 weeks of full-time teaching (as assumed here), or 2 weeks of capstone build at the end of a longer module?
- [ ] Module code, credit value and which year and specializations it is for.
- [ ] Who pays for Claude and OpenAI access: student plans, a faculty education plan, or shared API keys with spending caps?
- [ ] Team size: 4 is assumed; with 3 per team, drop real-time chat or the integrations row from the Pass column.

- [ ] Is CampusOne the required project, or may teams always choose their own domain?
- [ ] Does the faculty allow Pass/Fail on criterion 9 alone (e.g. a failed viva fails the module)?